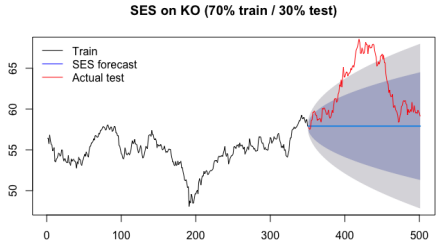
Simple Exponential Smoothing

From intuition and derivation to forecasting

Time Series Analysis

HEM Bellyday
SOPHON Rachana
TAING Korngmeng

Institute of Technology of Cambodia (ITC)



1 What is Simple Exponential Smoothing?

SES estimates the current level of a series by a weighted average of past observations, where weights decrease exponentially with age.

Why smooth?

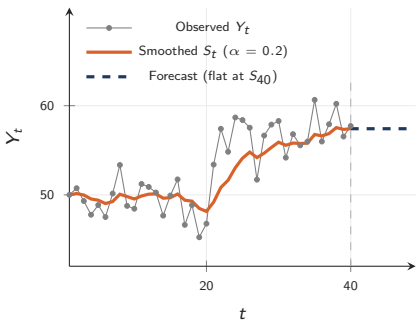
- Separate the *signal* (level) from *noise*
- Get a stable basis for forecasting

When is SES appropriate?

- No clear trend
- No seasonality
- Level may drift slowly over time

Example: weekly demand for rice or bottled water, fluctuating around a stable level.

Noisy level vs. smoothed level

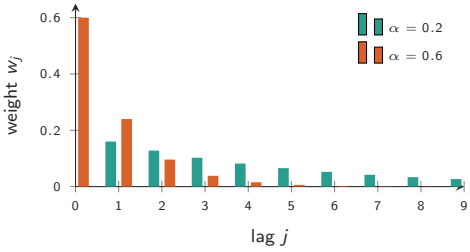


Simulated: level 50 → 58 at $t = 21$, noise sd 2.5. Left of $t = 40$: smoothing the past. Right: forecasting (flat).

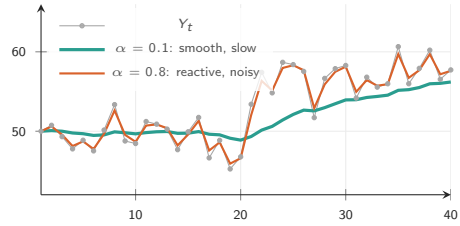
2 Theory and intuition: exponentially decaying weights

Weight on the observation j periods ago:

$$w_j = \alpha\,(1-\alpha)^j, \qquad 0 < \alpha \leq 1$$



Same data, two values of α



	Small α	Large α
Memory	long	short
Smoothing	strong	weak
Adapts to change	slowly	quickly

Random errors are averaged over many past points, so they partly cancel; a persistent change in level keeps pulling S_t until it catches up.

3 Recursive formula and derivation

$$S_t = \alpha\, Y_t + (1-\alpha)\, S_{t-1}, \quad t \geq 2$$

- Y_t observation at time t
- S_t smoothed value (level) at t
- α smoothing parameter, $0 < \alpha \leq 1$
- S_1 initial value (e.g. $S_1 = Y_1$)

Why exponential? Each step back multiplies the weight by the same factor $(1-\alpha) < 1$ (geometric decay).

Weights sum to one, so S_t is a true weighted average:

$$\alpha \textstyle\sum_{j=0}^{t-2}(1-\alpha)^j + (1-\alpha)^{t-1} = 1$$

Substitute $S_{t-1} = \alpha Y_{t-1} + (1-\alpha)S_{t-2}$, and so on:

$$\begin{aligned} S_t &= \alpha Y_t + (1-\alpha)S_{t-1} \\ &= \alpha Y_t + (1-\alpha)\big[\alpha Y_{t-1} + (1-\alpha)S_{t-2}\big] \\ &= \alpha Y_t + \alpha(1-\alpha)Y_{t-1} + (1-\alpha)^2 S_{t-2} \\ &\;\;\vdots \\ &= \underbrace{\alpha \sum_{j=0}^{t-2}(1-\alpha)^j\, Y_{t-j}}_{\text{weighted data}} + \underbrace{(1-\alpha)^{t-1} S_1}_{\text{initial value}} \end{aligned}$$

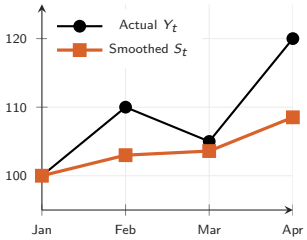
Role of S_1: its weight $(1-\alpha)^{t-1} \to 0$ as t grows. It matters a lot for short series or small α, little for long series.

4 Manual calculation: $\alpha = 0.3, \quad S_1 = 100$

Month	Y_t	$S_t = 0.3\,Y_t + 0.7\,S_{t-1}$	S_t
January	100	initial value	**100.00**
February	110	$0.3(110) + 0.7(100) = 33 + 70$	**103.00**
March	105	$0.3(105) + 0.7(103) = 31.5 + 72.1$	**103.60**
April	120	$0.3(120) + 0.7(103.6) = 36 + 72.52$	**108.52**

Check with the expanded formula ($t = 4$):

$$S_4 = 0.3\,[\,120 + 0.7(105) + 0.7^2(110)\,] + 0.7^3(100)$$
$$= 0.3(247.4) + 34.3 = 74.22 + 34.3 = \mathbf{108.52}\ \checkmark$$



Rule of thumb: keep 30% of the new value and 70% of the old level.

5 Forecasting with SES

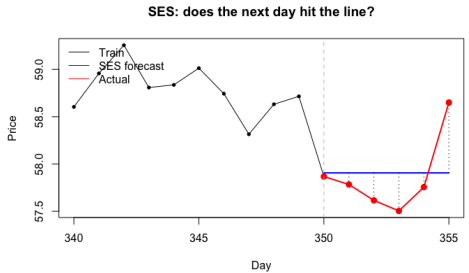
One step ahead: forecast = latest level

$$\hat{Y}_{T+1|T} = S_T \;\Rightarrow\; \hat{Y}_{\text{May}|\text{Apr}} = S_4 = \mathbf{108.52}$$

***h* steps ahead:** flat forecast,
$\hat{Y}_{T+h|T} = S_T,\ h = 1, 2, \ldots$

Why flat? Future Y's are unknown; replacing Y_{T+1} by its forecast S_T gives $\alpha S_T + (1-\alpha)S_T = S_T$. With no trend or season there is nothing to make the level move.

Quantity	Notation	Uses data up to
Actual observation	Y_t	— (measured)
Smoothed value (level)	S_t	time t (includes Y_t)
Fitted value	$\hat{Y}_{t\|t-1} = S_{t-1}$	time $t-1$ only
Forecast	$\hat{Y}_{T+h\|T} = S_T$	end of sample T

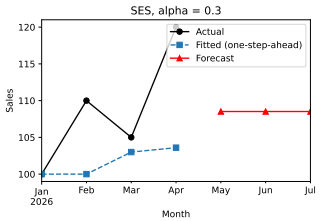


Coca-Cola (KO) daily close, downloaded with R `quantmod`.

Blue: flat SES forecast; dotted lines: forecast errors $Y_t - \hat{Y}_{t|T}$.

6 Practical implementation in Python (statsmodels)

```python
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.holtwinters import SimpleExpSmoothing
# (1) Data with a monthly date index
idx = pd.date_range("2026-01-01", periods=4, freq="MS")
sales = pd.Series([100, 110, 105, 120], index=idx)
# (2) Fix the start level and alpha (no estimation)
model = SimpleExpSmoothing(sales,
            initialization_method="known",
            initial_level=100)
fit = model.fit(smoothing_level=0.3, optimized=False)
# (3) Smoothed values, fitted values, forecasts
print(pd.DataFrame({"Y": sales, "S": fit.level,
                    "fitted": fit.fittedvalues}).round(2))
fc = fit.forecast(3)              # May, Jun, Jul
print(fc.round(2))                # 108.52 each
# (4) Plot actual, fitted and forecast
ax = sales.plot(marker="o", color="k", label="Actual")
fit.fittedvalues.plot(ax=ax, ls="--", marker="s",
                      label="Fitted (one-step-ahead)")
fc.plot(ax=ax, marker="^", color="r", label="Forecast")
ax.set(title="SES, alpha = 0.3", xlabel="Month",
       ylabel="Sales")
ax.legend(); plt.tight_layout(); plt.show()
```



Output	Y_t	level S_t	fitted
Jan	100	100.00	100.0
Feb	110	103.00	100.0
Mar	105	103.60	103.0
Apr	120	108.52	103.6
May–Jul		forecast	**108.52**

`initial_level` is the level *before* January (ℓ_0). Since $Y_1 = 100 = \ell_0$, January's level is $0.3(100) + 0.7(100) = 100 = S_1$, exactly our hand setup.

7 Advantages, limitations, and conclusion

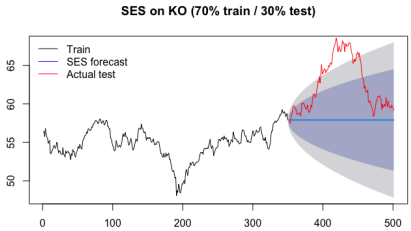
Advantages: one parameter; $O(1)$ update per point; easy to explain; α tunes responsiveness.

Limitations: no explicit trend or seasonality, so forecasts lag a trend and miss seasonal peaks; flat forecasts.

Method	Components	Forecast shape
SES	level (α)	flat line
Holt linear	level + trend (α, β)	straight line
Holt–Winters	level + trend + season (α, β, γ)	repeating pattern

Key takeaways

1. SES is a weighted average with weights $\alpha(1-\alpha)^j$.
2. α trades smoothness against responsiveness.
3. Forecast $= S_T$ for every horizon: use SES for level-only series.



KO via `quantmod`: the test path (red) rallies far above the flat forecast, even leaving the 95% band.

Question: Looking at KO, would *Holt's* method have done better here, or worse? Why?

Backup: KO example in R with `quantmod`

```r
library(quantmod); library(forecast)
getSymbols("KO", src = "yahoo", from = "2023-01-01") # adjust dates
y     <- as.numeric(Cl(KO))[1:500]                    # daily closing prices
n_tr  <- floor(0.7 * length(y))                       # 70% train / 30% test
train <- y[1:n_tr];  test <- y[(n_tr + 1):length(y)]

fit <- ses(train, h = length(test))   # alpha and initial level estimated
fit$model$par                         # inspect estimated alpha
plot(fit, main = "SES on KO (70% train / 30% test)")
lines((n_tr + 1):length(y), test, col = "red")
accuracy(fit, test)                   # train vs test error measures
```

For random-walk-like prices the estimated α is usually close to 1, so SES behaves almost like the naive forecast "tomorrow = today".